Manejo de Archivos en Python 🌟

Trabajar con archivos en `Python` nos permite leer y guardar información de forma permanente. Esto es fundamental cuando necesitamos que nuestros programas recuerden datos entre una ejecución y otra, procesen grandes cantidades de texto o generen reportes. En esta lección, vamos a explorar cómo interactuar con archivos de texto utilizando herramientas de `Python`.

Para todos los ejemplos de esta lección, vamos a usar un archivo llamado `quijote.txt` que tiene el siguiente contenido:

```
1 En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía
2 Una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sá
3 El resto della concluían sayo de velarte, calzas de velludo para las fiestas, con sus pant
```

Copiar

Modos de Apertura 📂

Para leer o escribir un archivo en `Python`, el primer paso es utilizar la función incorporada `open()`. Esta función se encarga de establecer una conexión entre nuestro programa y el archivo en el disco duro.

Al usar `open()`, debemos especificar dos argumentos principales: la **ruta del archivo** y el **modo de apertura**. El **modo de apertura** define exactamente qué operaciones tenemos permitido realizar (*leer, escribir, modificar, etc.*). Si no especificamos ningún modo, se asume por defecto que solo queremos leer el archivo (`'r'`).

A continuación, se presentan los modos de apertura más utilizados:

Modo	Acción	¿Sobrescribe el contenido?	¿Crea el archivo si no existe?
`'r'`	Lectura (*read*)	❌	❌
`'w'`	Escritura (*write*)	✅	✅
`'a'`	Agregar al final (*append*)	❌	✅
`'x'`	Crear archivo nuevo (*falla si ya existe*)	❌	✅

La sintaxis básica para abrir un archivo es la siguiente:

```
1  with open('quijote.txt', 'r') as file:
2      content = file.read()
```

Copiar

En este bloque incluimos la declaración `with` la cual nos asegura que el archivo se cerrará automáticamente cuando el bloque de código termine, incluso si ocurre un error inesperado, evitando problemas de memoria o bloqueos de archivos en el sistema. Es posible abrir archivos sin este bloque, pero una excelente práctica es usarlo para asegurar que el archivo se cierre de forma adecuada.

Lectura de Archivos 📖

Una vez que hemos abierto un archivo en modo lectura (`'r'`), tenemos varias formas de extraer su contenido. Las dos más comunes son `read()` y `readlines()` .

Extraer todo el contenido 🏃

El método `read()` toma todo el texto dentro del archivo y lo devuelve como una única cadena de texto (*`string`*). Esto es muy útil cuando el archivo es de tamaño manejable y necesitamos analizar todo el contenido a

la vez.

Para nuestros ejemplo, creamos el archivo `quijote.txt` en la misma carpeta que nuestro código, el cual contiene las primeras líneas del libro.

```python
def read_complete_file(path):
    # Usamos 'with' para un manejo seguro del archivo
    with open(path, 'r', encoding='utf-8') as file:
        content = file.read()
        print(content)

read_complete_file('quijote.txt')
```

Copiar

En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor.
Una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lantejas los viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.
El resto della concluían sayo de velarte, calzas de velludo para las fiestas, con sus pantuflos de lo mesmo, y los días de entresemana se honraba con su vellorí de lo más fino.

Leer línea por línea 🚶

Cuando un archivo es extenso, leerlo todo de una vez puede consumir demasiada memoria. La función `readlines()` permite leer el archivo completo pero devuelve una lista, donde cada elemento es una línea del archivo. Esto nos facilita iterar sobre el texto, procesando línea por línea.

Copiar

```python
def read_file_by_lines(path):
    with open(path, 'r', encoding='utf-8') as file:
        lines = file.readlines()

        # Iteramos sobre la lista de líneas obtenida
        for number, line in enumerate(lines, start=1):
            # Usamos strip() para remover los saltos de línea y limpiar espacios
            print(f"Line {number}: {line.strip()}")

read_file_by_lines('quijote.txt')
```

Line 1: En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor.
Line 2: Una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lantejas los viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.
Line 3: El resto della concluían sayo de velarte, calzas de velludo para las fiestas, con sus pantuflos de lo mismo, y los días de entresemana se honraba con su vellorí de lo más fino.

Escritura de Archivos ✍🏻

Para escribir información en un archivo, usamos el método `write()`. En este caso, el comportamiento depende del modo de apertura que hayamos elegido: `'w'` o `'a'`.

Sobrescribir contenido con modo `'w'` 🖍️

El modo de escritura (`'w'`) creará el archivo si no existe, o **borrará todo su contenido previo** si ya existe, antes de comenzar a escribir. Debe usarse con extrema precaución.

```python
def write_new_file(path, text):
    with open(path, 'w', encoding='utf-8') as file:
        file.write(text)

new_text = "Capítulo II. Que trata de la primera salida que de su tierra hizo el ingenioso

write_new_file('quijote_capitulo2.txt', new_text)
```

Copiar

Agregar contenido con modo `'a'` 📈

Si queremos mantener la información existente y solo añadir texto nuevo al final, debemos usar el modo de adición o *append* (`'a'`).

```python
def append_to_file(path, extra_text):
    with open(path, 'a', encoding='utf-8') as file:
        # Añadimos un salto de línea antes del nuevo texto para no pegarlo al anterior
        file.write("\\n" + extra_text)

additional_text = "Hechas, pues, estas prevenciones, no quiso aguardar más tiempo a poner

append_to_file('quijote_capitulo2.txt', additional_text)
```

Copiar

Rutas Absolutas y Relativas 🚩

Este es un buen momento para dar un pequeño repaso a cómo le indicamos a `Python` dónde encontrar nuestros archivos. Como hemos mencionado en la lección de terminales anteriormente, tenemos dos opciones:

Rutas Absolutas: Describen la ubicación completa desde la raíz del sistema.

Rutas Relativas: Describen la ubicación partiendo desde donde estamos ejecutando el código.

Veamos un ejemplo de cómo se verían en código:

```
1  # Ejemplo usando una ruta absoluta en Windows
2  # Evitamos este tipo de ruta para que nuestro código sea portable
3  absolute_file = open('C:\\Users\\Documents\\Projects\\quijote.txt')
4
5  # Ejemplo usando una ruta relativa
6  # Se busca en la misma carpeta donde ejecutamos el script
7  relative_file = open('quijote.txt')
```

Copiar

💡 En nuestro entorno de trabajo siempre daremos preferencia a las **rutas relativas** para que nuestro código sea fácil de compartir y ejecutar en cualquier computadora sin tener que reescribir direcciones completas.

Codificación (*Encoding*) 👨‍💻

Las computadoras solo entienden números, por lo que utilizan tablas de codificación para traducir texto legible a ceros y unos. Cuando trabajamos con idiomas que incluyen caracteres especiales como tildes, la letra `ñ` o símbolos de apertura de interrogación (`¿`), es crucial especificar la codificación correcta al abrir o guardar un archivo.

Si al leer o escribir un archivo en `Python` no definimos la codificación explícitamente, el sistema operativo intentará usar la suya por defecto. Esto usualmente resulta en caracteres corruptos, mostrando símbolos extraños en la consola.

Veamos un ejemplo de lo que ocurre si ignoramos la codificación al intentar guardar un texto con caracteres especiales:

```
def save_corrupted_text(path, text):
    # Guardamos sin especificar encoding.
    with open(path, 'w') as file:
        file.write(text)

text_with_accents = "Algún palomino de añadidura los domingos"

save_corrupted_text('bad_encoding.txt', text_with_accents)
```

Copiar

Si otro programa o sistema intenta abrir este archivo asumiendo un formato distinto, los caracteres especiales se rompen y el resultado se ve así:

```
Alg�n palomino de a�adidura los domingos
```

Copiar

Para evitar esto, siempre debemos especificar el parámetro `encoding='utf-8'` dentro de la función `open()`. `UTF-8` es el estándar de codificación global que soporta prácticamente cualquier carácter de la gran mayoría de los idiomas.

```python
def save_secure_text(path, text):
    # Agregamos encoding='utf-8' para manejar tildes de manera correcta
    with open(path, 'w', encoding='utf-8') as file:
        file.write(text)

text_with_accents = "En resolución, él se enfrascó tanto en su letura, que se le pasaban l

save_secure_text('quijote_notas.txt', text_with_accents)
```

Copiar

En este último caso, si abrimos `quijote_notas.txt`, veremos las palabras "*resolución*", "*enfrascó*" y "*días*" bien formadas, garantizando que el texto se mantiene íntegro ante cualquier entorno de desarrollo o sistema operativo en donde ejecutemos nuestro código.

Conclusión 🎉

A lo largo de esta lección hemos aprendido a gestionar el ciclo completo de vida de un archivo de texto en `Python`. Sabemos cómo emplear la instrucción `open()` junto con `with` para una gestión eficiente de la memoria, podemos escribir o leer contenido utilizando los diversos modos. También comprendemos la importancia vital de utilizar un *encoding* adecuado para evitar corromper caracteres.

Con estos fundamentos bien establecidos, estamos listos para empezar a construir aplicaciones que guarden configuraciones, procesen grandes registros de texto y administren su propia persistencia de datos. 💻